Number Theory

Antarjot Singh

ANCC from IIT Delhi

Contents of the Slides/Lecture

Fundamentals (Divisibility)

- **What do we mean by divides:** $\rightarrow$ Let a and b be integers ($a, b \in \mathbb{Z}$). We say that a divides b (or that b is a multiple of a, or that a is a divisor of b) if and only if there exists an integer $c \in \mathbb{Z}$ such that

$$b = a \cdot c.$$

- **Notation and Terminology:**
  - Notation: $a \mid b$ (read as "a divides b").
  - Negation: $a \nmid b$ (read as "a does not divide b"), which means there is no integer c such that

$$b = a \cdot c.$$

Fundamentals (Divisibility Contd.)

Properties of Divisibility

- **Trivial Divisors:**
  - $1 \mid a$ and $-1 \mid a$ for all $a \in \mathbb{Z}$.
  - $a \mid 0$ for all $a \in \mathbb{Z}$.
- **Reflexivity:**

$$a \mid a \qquad \forall a \in \mathbb{Z}$$

- **Transitivity:**

$$\text{If } a \mid b \text{ and } b \mid c, \text{ then } a \mid c.$$

- **Multiplicative Scaling:**
  - If $a \mid b$, then $a \mid (bc)$.
  - If $a \mid b$, then $ac \mid bc$.
- **Cancellation Property:**

$$\text{If } (ca) \mid (cb),\ c \neq 0 \Rightarrow a \mid b.$$

- **Addition/Subtraction Property:**

$$a \mid b,\ a \mid c \Rightarrow a \mid (b + c),\ a \mid (b - c).$$

Fundamentals (Division Algorithm and Modulo)

- **Division Algorithm**
  For integers a, b with $b > 0$, there exist unique integers q, r such that

$$a = bq + r, \qquad 0 \leq r < b$$

  - If $r = 0$, then $a = bq$ and hence $b \mid a$
  - If $r \neq 0$, then $b \nmid a$
- **Programming View**
  - Integer division: $q = a/b$
  - Modulo operator: $r = a\%b$
- **Negative Number Trap**
  - Mathematics/ python: -5=3(-2)+1
  - C++ / Java: -5%3=-2
- **Correct Non-negative Modulo**

```
int remainder = (a%b+b)%b;
```

Fundamentals (Fundamental Theorem of Arithmetic)

- **The Theorem**
  Every integer $n > 1$ can be represented as a product of prime numbers.
  This representation is unique, up to the order of the prime factors.

- **Canonical Form**

$$n = p_1^{a_1} \cdot p_2^{a_2} \cdot p_3^{a_3} \cdots p_k^{a_k}$$

  where

$$p_1 < p_2 < \cdots < p_k$$

  are distinct prime numbers and a_i are positive integers.

- **Example** $120 = 2^3 \cdot 3^1 \cdot 5^1$
  No matter how we factorize 120, we always obtain exactly three 2s, one 3, and one 5.
- For any positive integer n and any prime p, the n-th root $\sqrt[n]{p}$ is unconditionally irrational.

Fundamentals (Binary Exponentiation)

- Lets say for now multiplication takes $O(1)$ time. Now we will come across several problems where you have compute x^k then our total complexity will be $O(k)$ times.
- Sometimes we can't afford this much time, so we use a technique called **Binary exponentiation**.
- **What is Binary Exponentiation?**

  Binary Exponentiation is a fast method to calculate large powers such as a^n in logarithmic time.
- **Main Observation**

  Instead of multiplying a repeatedly, we use the fact that powers can be using Binary representation of any number and squaring the answer. $a^8 = ((a^2)^2)^2$

Fundamentals (Binary Exponentiation Contd.)

- **Algorithm** You have to compute x^n and base power case is if $n = 0$ then return 1 At every step:
  - If the exponent is odd, multiply the current answer by the base.
  - Square the base.
  - Halve the exponent.
- **Why is it Fast?**

  Normal multiplication needs n operations.

  Binary Exponentiation reduces the exponent by half each step, so it only needs $O(\log n)$ operations.
- **Example Idea** To compute 2^{13} we repeatedly square:

$$2,\ 4,\ 16,\ 256, \ldots$$

  and only multiply the required powers together.

Fundamentals (Binary Exponentiation Contd.)

Recursive Code

```
ll binpow(ll a,ll n,ll mod){
    if(n==0)
        return 1;
    ll x=binpow(a,n/2,mod);
    x=(x*x)%mod;
    if(n&1)
        x=(x*a)%mod;
    return x;
}
```

Iterative Code

```
ll binpow(ll a,ll n,ll mod){
    ll ans=1;
    while(n){
        if(n&1)
            ans=(ans*a)%mod;
        a=(a*a)%mod;
        n>>=1;
    }
    return ans;
}
```

Fundamentals (Euclidean Algorithm)

- **Definition**

  Let $a, b \in \mathbb{Z}$, not both zero.

  The **Greatest Common Divisor (GCD)** of a and b is the largest positive integer d such that $d \mid a$ and $d \mid b$.

  We denote it by $\gcd(a, b)$ or (a, b). Replacing the larger number by its remainder when divided by the smaller number does not change the GCD.

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

- **Algorithm** We Repeatedly apply $(a, b) \to (b, a \bmod b)$ until the remainder becomes 0.
- **Stopping Condition** If $a = b \cdot q + 0$ then $\gcd(a, b) = b$.

- **Complexity/No. of Operations** $O(\log(\min(a, b))) \to$ **WHY?** because

$$x \bmod y \leq \frac{x}{2}$$

Fundamentals (Euclidean Algorithm Contd.)

- **Symmetry:** $\gcd(a, b) = \gcd(b, a)$
- **Euclidean Property:** $\gcd(a, b) = \gcd(a, b - ka)$ for any integer k.
- **Scaling:** $\gcd(ka, kb) = k \cdot \gcd(a, b)$
- **Coprime Numbers** $\gcd(a, b) = 1$.
- **Bézout's Identity**
  There exist integers x, y such that $ax + by = \gcd(a, b)$.
- **Product Formula** $\gcd(a, b) \cdot \text{lcm}(a, b) = ab$
- CF 1458A

Recursive

```
ll gcd(ll a,ll b){
    if(b==0)
        return a;
    return gcd(b,a%b);
}
```

Iterative

```
ll gcd(ll a,ll b){
    while(b) {
        ll r=a%b;
        a=b;
        b=r;
    }
    return a;
}
```

Fundamentals (Extended Euclidean Algorithm)

- **Objective:** Bézout's Identity guarantees that for any integers a and b, we can find coefficients x and y such that:

$$ax + by = \gcd(a, b)$$

The Extended Euclidean Algorithm computes both $\gcd(a, b)$ and a valid pair (x, y) simultaneously.

- **Mathematical Recurrence Transitions:**
  - Base Case: When $b = 0$, $\gcd(a, 0) = a$. Thus, $a(1) + 0(0) = a \implies x_0 = 1, y_0 = 0$.
  - Step Update: Let $a\%b = a - \lfloor a/b \rfloor \cdot b$. Substituting this into the recursive state $b \cdot x_1 + (a\%b) \cdot y_1 = g$ yields the current state coefficients:

$$x = y_1, \qquad y = x_1 - \lfloor a/b \rfloor \cdot y_1$$

Recursive Approach

```
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

Iterative Approach

```
ll extgcd(ll a, ll b, ll &x, ll &y) {
    x = 1; y = 0;
    ll x1 = 0, y1 = 1;
    while (b) {
        ll q = a / b;
        ll r = a % b;
        a = b; b = r;

        ll next_x = x - q * x1;
        x = x1; x1 = next_x;

        ll next_y = y - q * y1;
        y = y1; y1 = next_y;
    }return a;}
```

Analysis of Primes (Factoring a Number)

- Problem Homework
- **What is a prime number:** A number is a prime number which is divisble by 1 and itself.
- **What is a Factor/Divisor of a number:** A number x is called a Factor/Divisor of a number y if $x \mid y$.
- **Core Observation:** Divisors always occur in symmetric pairs. If $d \mid N$, then $\frac{N}{d} \mid N$ as well.
- **The $\sqrt{N}$ Bound Rule:** In any factor pair $(d, \frac{N}{d})$, it is mathematically impossible for both factors to be strictly greater than $\sqrt{N}$.

$$\text{If } d > \sqrt{N} \text{ and } \frac{N}{d} > \sqrt{N} \implies d \cdot \frac{N}{d} > \sqrt{N} \cdot \sqrt{N} = N \quad \text{(Contradiction)}$$

- Our Core objective is to check whether a number is Prime and find all factors of the given number.
- Naive Approach to calculate all the factors of a given number N will to be just check every number from $[1, N]$, but by using The $\sqrt{N}$ Bound Rule we only need to scan integers in the range $[1, \lfloor\sqrt{N}\rfloor]$ to uncover all divisors.
- If $d = \frac{N}{d}$ (i.e., N is a perfect square), the factor d must only be counted once to avoid duplicate entries.

Analysis of Primes (Factoring a Number Contd.)

Factors Collection Algorithm ($O(\sqrt{N})$)

```
vector<ll> get_factors(ll n) {
    vector<ll> divisors;
    for (ll d = 1; d * d <= n; ++d) {
        if (n % d == 0) {
            divisors.push_back(d); // Smaller factor

            if (d * d != n) {
                divisors.push_back(n / d); // Larger partner
            }
        }
    }
    return divisors;
}
```

Analysis of Primes (Sieve of Eratosthenes)

Imagine writing down numbers from 2 up to 100. Our goal is to find all the primes by systematically "knocking out" the numbers that are clearly not prime (the composites).

How the "Sieve" Works (Step-by-Step):

- **Start at 2:** Circle 2 (it's the first prime). Then, cross out every multiple of 2 (4, 6, 8, 10...) because they can all be divided by 2.
- **Move to 3:** Move to the next uncrossed number, which is 3. Circle it. Now, cross out all remaining multiples of 3 (9, 15, 21...) that aren't already crossed out.
- **Skip the Crossed-Outs:** The next number is 4, but it's already crossed out. Skip it.
- **Repeat for 5:** The next open number is 5. Circle it, and cross out all of its multiples (25, 35, 55...).

The Magic Secret: **Every single number left uncrossed at the end is guaranteed to be a prime number!**

Analysis of Primes (Sieve of Eratosthenes Contd.)

Easy Implementation

```
vector<int> primes;
vector<bool> is_prime(n+1,true);

is_prime[0]=is_prime[1]=false;

for(int i=2;i<=n;i++){

    if(!is_prime[i])
        continue;

    primes.push_back(i);

    for(int j=i+i;j<=n;j+=i)
        is_prime[j]=false;
}
```

Complexity Analysis

- For every prime p, we mark $\frac{n}{p}$ multiples.
- Total work:

$$n\left(\frac{1}{2}+\frac{1}{3}+\frac{1}{5}+\cdots\right)$$

- Using the fact that

$$\sum_{p\leq n}\frac{1}{p}=O(\log\log n)$$

we obtain

$$T(n)=O(n\log\log n)$$

- Memory Usage:

$$O(n)$$

for the boolean array.

Practice Problem

Problem

You are given two integers n and q (maximum limit and number of queries).
For each query k, print the count of integers in the range $[1, n]$ having exactly k distinct prime factors.

Analysis of Primes (Logarithmic Factorization via SPF)

Why do we need SPF (Small Prime Factor)?

Suppose we need to factorize a single number N. Using trial division, we may need to test all numbers up to $\sqrt{N}$ which takes $O(\sqrt{N})$ time.

However, if we need to answer many factorization queries, this becomes too slow.

How Factorization Works

To factorize N:

- Look up SPF(N).
- Add it to the factorization.
- Divide N by SPF(N).
- Repeat until $N = 1$.

Example: $60 \to 2 \to 30 \to 2 \to 15 \to 3 \to 5 \to 5 \to 1$ Thus,

$$60 = 2^2 \cdot 3^1 \cdot 5^1.$$

What Do We Obtain?

From the SPF chain we can recover:

- All prime factors.
- The exponent (power) of each prime.
- The complete prime factorization of N.

Analysis of Primes (Logarithmic Factorization via SPF Contd.)

C++ Implementation

```
const int MAXN=1000001;
int spf[MAXN];
void build_spf(){
    for(int i=1;i<MAXN;i++)
        spf[i]=i;
    for(int p=2;p*p<MAXN;p++){
        if(spf[p]!=p)
            continue;
        for(int j=p*p;j<MAXN;j+=p)
            if(spf[j]==j)
                spf[j]=p;
    }
}
vector<int> factorize(int n){
    vector<int> factors;
    while(n>1){
        factors.push_back(spf[n]);
        n/=spf[n];
    }
    return factors;
}
```

What does factorize() return?
For $60 = 2^2 \cdot 3 \cdot 5$ the function returns

$$[2, 2, 3, 5].$$

From this list we can easily obtain

- Prime factors
- Distinct prime factors
- Prime powers

Complexity

- Building SPF: $O(M \log \log M)$ where $M = \texttt{MAXN}$.
- Factorizing one number: $O(\log N)$ because each step divides N by a prime factor ≥ 2 worst case.
- Memory:

$$O(M)$$

for storing the SPF array.

Analysis of primes (The Linear Sieve)

The Problem with the Standard Sieve: When filtering numbers, the standard Sieve does redundant work. For example, the number 12 gets visited and marked false by 2 (since $2 \times 6 = 12$) and then visited *again* by 3 (since $3 \times 4 = 12$). As N grows into the millions, this duplicate marking slows us down.
The Linear Sieve Rule (Cross Out Exactly Once): The Linear Sieve eliminates duplicate work with one strict rule:

*"Every composite number must be crossed out by its **smallest prime factor** only."*

How it works step-by-step:

- We maintain an active, growing list of discovered prime numbers.
- For every number i (whether it is prime or composite), we pair it up with the primes we have found so far to cross out new numbers ($i \times$ prime).
- **The Break:** The moment i becomes divisible by the current prime we are using to multiply, **we immediately stop**!
- Why stop? Because any future multiplications would use a larger prime, violating our rule of only using the *smallest* prime factor.

Analysis of primes (The Linear Sieve Contd.)

C++ Implementation ($O(N)$)

```
vector<int> primes;
vector<bool> is_prime(MAXN, true);

void linear_sieve(int n) {
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
        }
        // Multiply current i with found primes
        for (int p : primes) {
            if (p * i > n) break;

            is_prime[i * p] = false;

            // The CRITICAL Magic Stop condition
            if (i % p == 0) break;
        }
    }
}
```

Rigorous Analysis

- **Why `if (i % p == 0) break;` works:** If i is divisible by p, we can write $i = p \times k$. If we didn't break and moved to the next prime p_{next}, we would mark the number $x = i \times p_{next} = (p \times k) \times p_{next}$. Notice that the smallest prime factor of x is p, not p_{next}! Therefore, x should wait to be marked later when our loop index reaches $(k \times p_{next})$.
- **Time Complexity:** Strict $\mathcal{O}(N)$. Since every composite number is visited exactly once, the inner loop operations across the entire execution scale completely linearly with N.
- **Space Complexity:** $\mathcal{O}(N)$ to store the boolean array and the list of primes.

Analysis of Primes (Problem)

Problem

Find the number of unique ordered pairs (x, y) such that:

$$x + y = N \quad \text{and} \quad \gcd(x, y) = 1$$

The Mathematical Insight: By the Euclidean property, $\gcd(x, y) = \gcd(x, N - x)$. Since $\gcd(A, B) = \gcd(A, B \pmod A)$, it follows:

$$\gcd(x, N - x) = \gcd(x, N) = 1$$

The Revelation: Finding a valid pair (x, y) is exactly the same as finding an integer x in the range $[1, N]$ such that $\gcd(x, N) = 1$. This is the definition of the **Euler Totient Function**.

Analysis of Primes (Euler totient function)

The function $\phi(n)$ counts the number of integers k in the range $1 \leq k \leq n$ for which $\gcd(k, n) = 1$.
If $n = p_1^{a1} \cdot p_2^{a2} \cdots p_k^{ak}$, then:

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Properties:

- If p is prime, $\phi(p) = p - 1$.
- If $\gcd(a, b) = 1$, then $\phi(a \cdot b) = \phi(a) \cdot \phi(b)$ (Multiplicative).
- If $p|i$, then $\phi(i \cdot p) = \phi(i) \cdot p$.

Analysis of Primes (Euler totient function Contd.)

C++ Global Linear Pass ($O(N)$)

```cpp
vector<int> primes;
int phi[MAXN];
bool is_prime[MAXN];
void compute_all_phi(int n) {
    fill(is_prime, is_prime + n + 1, true);
    is_prime[0] = is_prime[1] = false;
    phi[1] = 1; // Base Case: phi(1) = 1
    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);phi[i] = i - 1;
        }
        for (int p : primes) {
            if (i * p > n) break;
            is_prime[i * p] = false;
            if (i % p == 0) {
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
}
```

Rigorous Mathematical Transitions

- **Case 1: i is Prime**
  By fundamental definition, a prime number shares no factors with anything smaller than it.
  $\implies \phi(i) = i - 1$.
- **Case 2:** `i % p != 0`
  Since p does not divide i, $\gcd(i, p) = 1$. Utilizing the strict multiplicative property of the totient function:

$$\phi(i \cdot p) = \phi(i) \cdot \phi(p) = \phi(i) \cdot (p - 1)$$

- **Case 3:** `i % p == 0`
  Here, p already divides i. This means $i \cdot p$ contains the exact same set of unique prime factors as i, only with an incremented exponent power. No new structural fractions are introduced to the product formula:

$$\phi(i \cdot p) = \phi(i) \cdot p$$

- **Time Complexity:** $\mathcal{O}(N)$
  The magic condition `if (i % p == 0) break;` guarantees that every composite number is visited and updated **exactly once** by its unique smallest prime factor. No redundant operations occur.